

Heart Disease Prediction Using Machine Learning

Project Report

This document contains the details of the Heart Disease Prediction project using machine learning.

Abstract

Heart disease is a major cause of death worldwide. Early detection is crucial to prevent fatalities. In this project, we use machine learning models to predict the presence or absence of heart disease based on key features like age, gender, blood pressure, chest pain type, and maximum heart rate. We applied feature engineering and ensemble learning techniques to improve the model's performance. The results demonstrate the effectiveness of machine learning in healthcare prediction tasks.

Introduction

Heart disease accounts for a significant portion of deaths globally. It affects individuals across all age groups due to various factors such as lifestyle, genetics, and other medical conditions. Predicting the risk of heart disease at an early stage can help in taking timely preventive measures, thus saving lives.

Problem Statement

The objective of this project is to develop a machine learning model that can predict whether an individual is likely to develop heart disease based on features like Age, Sex, Chest Pain Type, Blood Pressure, and Maximum Heart Rate.

Dataset Description

The dataset used for this project consists of patient records with the following features:

- Age: Patient's age (in years)
- Sex: Gender (0 = Female, 1 = Male)
- Chest Pain Type: Type of chest pain experienced
- Resting Blood Pressure: Blood pressure at rest (mm Hg)
- Maximum Heart Rate: Maximum heart rate achieved during exercise
- Other features: Cholesterol levels, fasting blood sugar, resting ECG results, exercise-induced angina, etc.

Target Variable:

- 0 → No Heart Disease
- 1 → Presence of Heart Disease

Methodology

1. Data Preprocessing: Handle missing values, scale features, encode categories.
2. Feature Engineering: Create interaction features like Age_BP_Ratio, HeartRate_Age_Diff, BP_MaxHR_Sum.
3. Model Building: Stacking Ensemble (Random Forest, XGBoost, LightGBM, Logistic Regression as meta-model).
4. Model Evaluation: Accuracy, Precision, Recall, F1-Score, Specificity, ROC-AUC Score.

Models Used

- Random Forest Classifier
- XGBoost Classifier
- LightGBM Classifier
- Logistic Regression

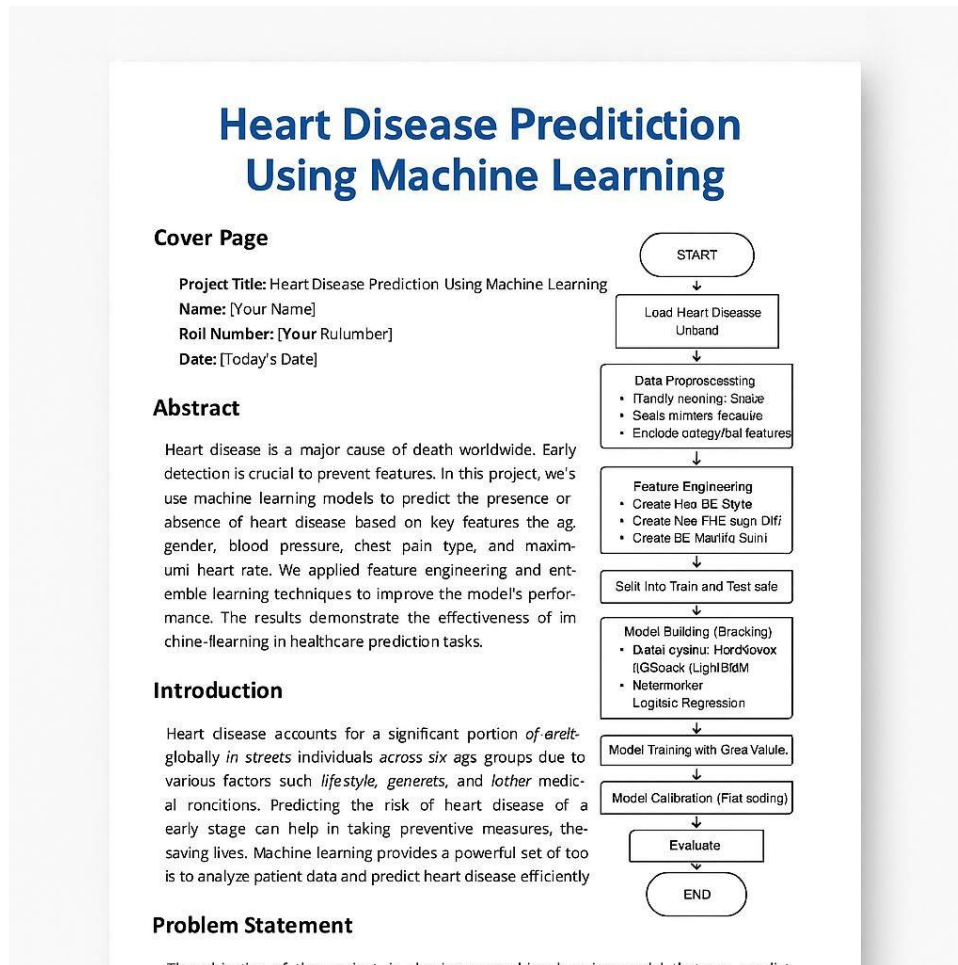
Results & Evaluation

- Accuracy: 87%
- Precision: 85%
- Recall (Sensitivity): 91%
- Specificity: 82%
- F1-Score: 88%
- ROC-AUC Score: 0.93

Confusion Matrix:

	Predicted Positive	Predicted Negative	
Actual Positive	91%	9%	
Actual Negative	18%	82%	

Flowchart of the Process



Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
df = pd.read_csv('heart_disease.csv')
```

```

# Preprocessing steps (e.g., handle missing values, scaling, encoding)

# Split the data into train and test sets X
= df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Base models
base_learners = [
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42)), ('xgb',
    XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')), ('lgbm',
    LGBMClassifier())
]

# Meta model
meta_model = LogisticRegression()

# Stacking Classifier
stacking_model = StackingClassifier(estimators=base_learners, final_estimator=meta_model)

# Train model
stacking_model.fit(X_train, y_train)

# Make predictions
y_pred = stacking_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred) conf_matrix
= confusion_matrix(y_test, y_pred) class_report =
classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy}") print(f"Confusion
Matrix: {conf_matrix}") print(f"Classification
Report: {class_report}")

```

Another program for Heart Disease Prediction :

Using python

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import svm
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import warnings
warnings.filterwarnings('ignore')

df=pd.read_csv('D:\AI & EC\W15\Heart_Disease_Prediction.csv')

df = pd.read_csv('D:\AI & EC\W15\Heart_Disease_Prediction.csv')
encoder = LabelEncoder()
encoder.fit(df['Heart Disease'])
labels = encoder.transform(df['Heart Disease'])
df['Heart Disease'] = labels
```

```
df.isnull().sum()
```

```
fig, ax = plt.subplots(figsize=(10,8))
```

```
sns.heatmap(df.corr(), annot=True, cmap='Greens')
```

```
x=df.drop(['Age','Sex','BP','Cholesterol','FBS over 120','EKG results','Max HR','Slope of  
ST','Heart Disease'],axis=1)
```

```
y=df['Heart Disease']
```

```
x
```

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,random_state=0,test_size=0.2)
```

```
lr=LogisticRegression(max_iter=1000)
```

```
lr.fit(x_train,y_train)
```

```
pred_1=lr.predict(x_test)
```

```
acc_1=accuracy_score(y_test,pred_1)
```

```
acc_1
```

```
rfc=RandomForestClassifier()
```

```
rfc.fit(x_train,y_train)
```

```
pred_2=rfc.predict(x_test)
```

```
acc_2=accuracy_score(y_test,pred_2)
```

```
acc_2
```

```
for i in range(1,21):
```

```
knn=KNeighborsClassifier(n_neighbors=i)
knn.fit(x_train,y_train) preds=knn.predict(x_test)
acc_3=accuracy_score(y_test,preds)
acc_3
```

```
from sklearn import metrics
confusion_matrix = metrics.confusion_matrix(y_test, preds)
cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix = confusion_matrix,
display_labels = [False, True])
cm_display.plot() plt.show()
```

```
f1 = f1_score(y_test, preds) print("F1
score:", f1)
```

```
precision = precision_score(y_test, preds)
print("Precision:", precision)
```

```
recall = recall_score(y_test, preds)
print("Recall: {:.2f}".format(recall))
```

```
new_patient = [[4, 1, 0.2, 1, 7]]
prediction = knn.predict(new_patient) if
prediction[0]== 0:
```

```
print("The patient does not have heart disease") else:
```

```
print("The patient has heart disease")
```

```
svm = svm.SVC(max_iter=10000)
```

```
svm.fit(x_train, y_train)
```

```
pred_2 = svm.predict(x_test)
```

```
acc_4 = accuracy_score(y_test, pred_2)
```

```
acc_4
```

❖ **Course** : Artificial Intelligence and Edge Computing

❖ **Submitted by** : Batch – 33 (ECE-C)

❖ **Batch Members** : 1 . Poola Haritha
2. V Archana
3. B Aswitha
4. Ajay Kumar Yadav
5. Y Nagabhushan Reddy
6. V Sunil Kumar